Describing data samples

Marcin Włodarczak

21 October 2024

Plan for today

The goal of this lab is to introduce some basic descriptive statistics which will be recurring in the remainder of this course.

Summarising a sample

We start by loading `tidyverse`:

```
library('tidyverse')
theme_set(theme_bw())
```

Today we're going to continue working with the `vowels_formants` data set from last time (remember to set the working directory to the location of this document!):

```
vowels <- read_csv('vowel_formants.csv')
```

Let's start by looking at the distribution of f_0. Before we do it, though, we might want to convert the values to semitones to allow for better comparisons between women and men. The formula for the conversion is

$$12 \times \log_2(\frac{f_0}{f_{0_{ref}}})$$

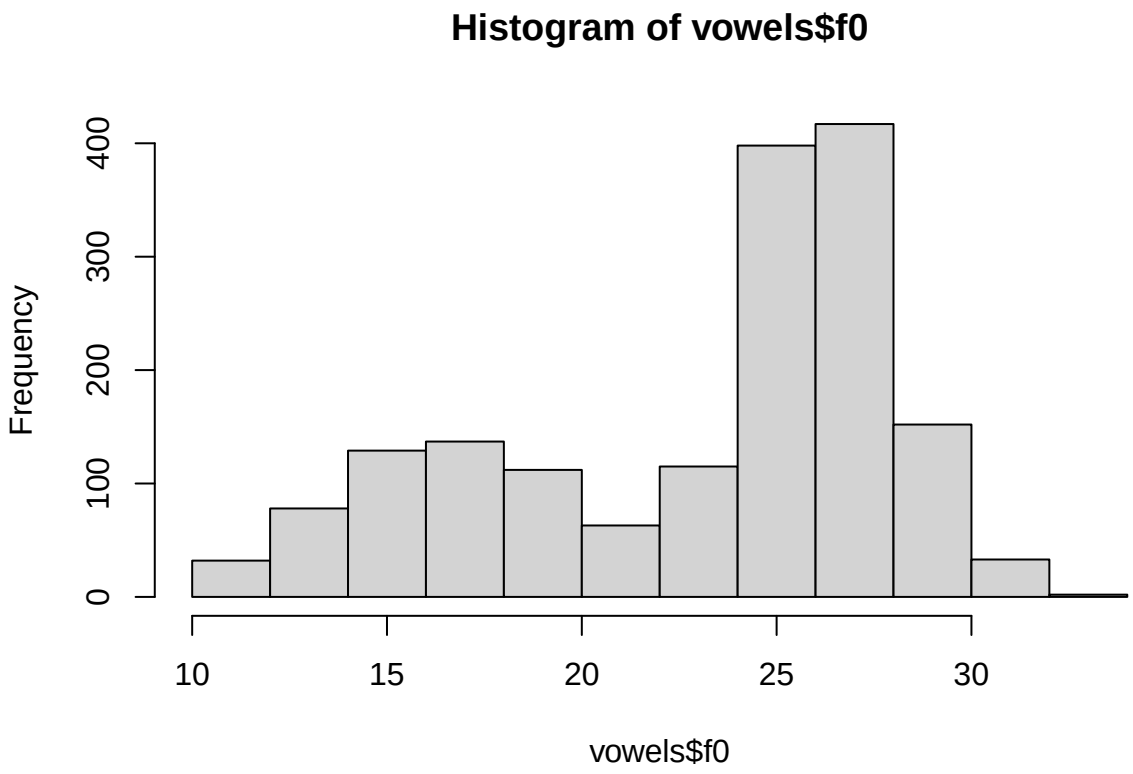
.

Using 50 Hz as a reference, this corresponds to the following line in R:

```
vowels <- mutate(vowels, f0 = 12 * log2(f0 / 50))
```

Now we can have a look at the *distribution* of f_0 values using a histogram, which shows how many data points fall into different data ranges (or *bins*):

```
hist(vowels$f0)
```



As you can see, we are getting really strange distribution with two distinct peaks (why?). Before we continue, let's filter all f_0 values in vowels produced by girls (we are using `pull` here to extract a column as a vector):

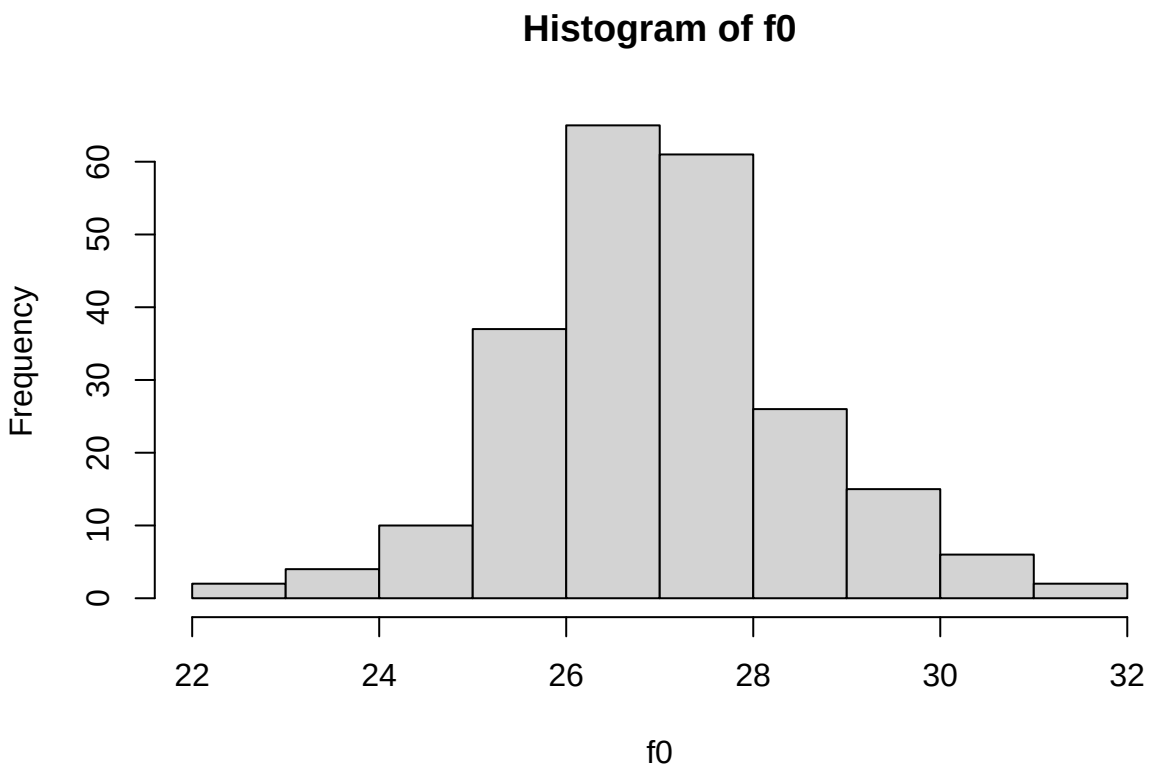
```
f0 <- vowels %>%
  filter(speaker == 'g') %>%
  pull(f0)

head(f0)
```

```
## [1] 27.30008 23.65024 28.20597 26.19231 25.80672 27.30008
```

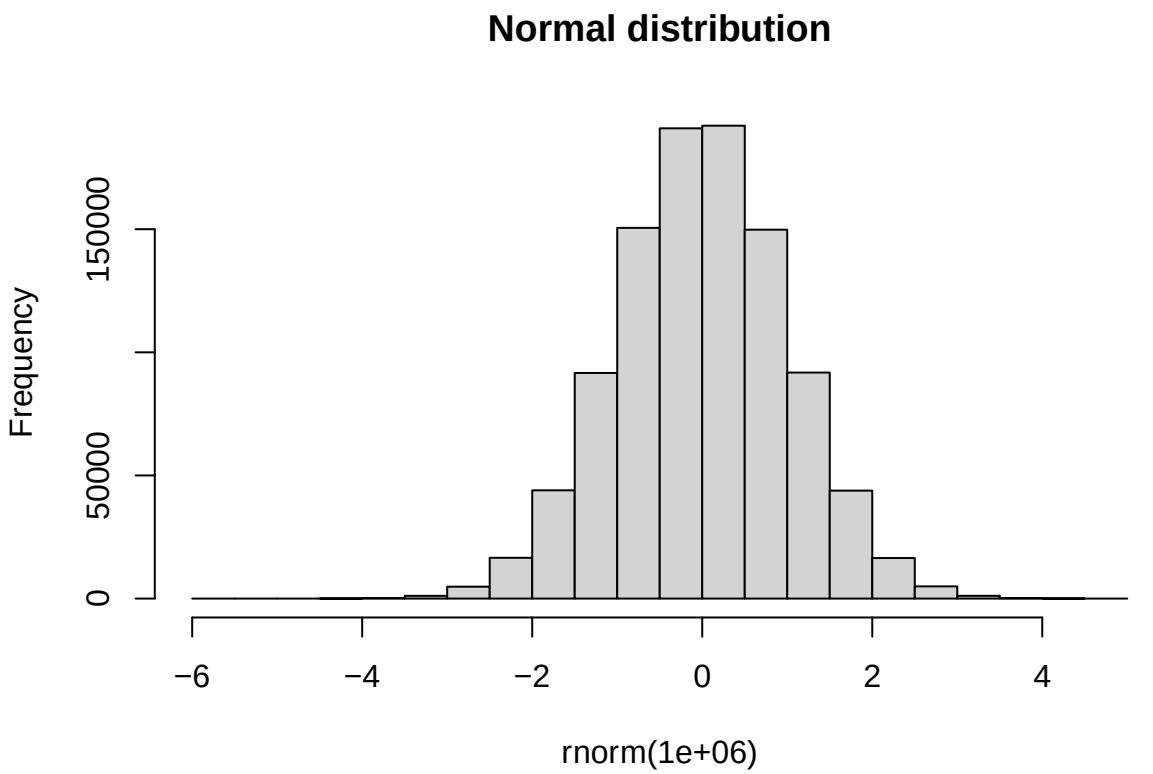
This looks much better:

```
hist(f0)
```



The reason why we like this kind of distribution is that the regression methods we are going to discuss in this course assume that many aspects of the data have a *normal distribution*, which looks like this:

```
hist(rnorm(1e6), main = 'Normal distribution')
```



The problem we are going to try to solve today is how can we summarise our little sample of f_0 values

numerically.

Central tendency: mean

The first thing we need to know average f_0. This is what the mean does:

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i = \frac{x_1 + x_2 + \ldots + x_{N-1} + x_N}{N}$$

That might look scary but, as the expansion on the right makes clear, it simply means: add up all the values and divide the sum by their number. This is easily done in R:

```
sum(f0) / length(f0)
```

```
## [1] 26.9717
```

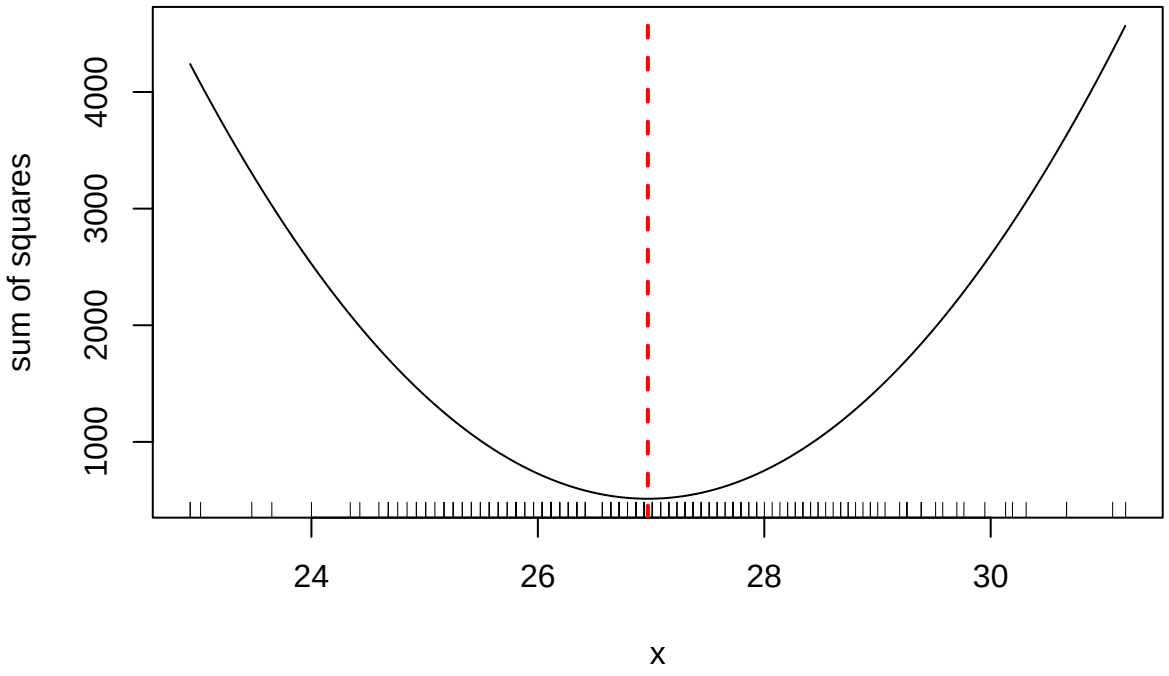
However, even that is not necessary since R comes with the `mean` function built-in:

```
mean(f0)
```

```
## [1] 26.9717
```

One interesting property of the mean is that it minimizes the sum of squared distances from the data points in the sample. In other words, if we pick a value, subtract it from each point in the sample, square the result (to get rid of negative values) and then sum everything up, the sum of squares will be smallest if the reference point we chose corresponds to the mean. Do not worry if the code below looks a bit cryptic, the important thing is to demonstrate the point:

```
x <- seq(min(f0), max(f0), 0.01)
sq_sum <- sapply(x, \(x) sum((f0 - x)^2))
plot(x, sq_sum, type = 'l', ylab = 'sum of squares')
rug(f0)
abline(v = mean(f0), lty = 2, col = 'red', lwd = 2)
```



What this means in practice is that the mean is our best guess about the expected value in the sample in the sense that it minimizes the error we would be making.

Before we move on, let's note one genuinely nice feature of `tidyverse`, that is the ability to create summaries for subgroups in the data. For instance, here is how we can calculate mean f_0 for each speaker group:

```
vowels %>%
  group_by(speaker) %>%
  summarise(f0_mean = mean(f0))
```

```
## # A tibble: 4 x 2
##   speaker f0_mean
##   <chr>     <dbl>
## 1 b          26.8
## 2 g          27.0
## 3 m          16.5
## 4 w          25.6
```

Similarly, we can calculate the mean f_0 for each group and vowel combination (we use `arrange` to sort by a specific column):

```
vowels %>%
  group_by(speaker, vowel) %>%
  summarise(f0_mean = mean(f0)) %>%
  arrange(vowel)
```

```
## `summarise()` has grouped output by 'speaker'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 48 x 3
## # Groups:   speaker [4]
##    speaker vowel f0_mean
##    <chr>   <chr>   <dbl>
##  1 b       ae       26.1
##  2 g       ae       26.6
##  3 m       ae       15.8
##  4 w       ae       25.1
##  5 b       ah       26.6
##  6 g       ah       26.1
##  7 m       ah       15.9
##  8 w       ah       24.9
##  9 b       aw       26.3
## 10 g       aw       26.7
## # i 38 more rows
```

Exercise

Find out whether women have higher formant (F1, F2) values than men.

```
# Write your solution here.
```

Exercise

We know that low vowels should have higher F1 values than high vowels. Check whether this is the case by comparing mean F1 in a low vowel (e.g. `"ah"`) and a high vowel (e.g. `"iy"`)

```
# Write your solution here.
```

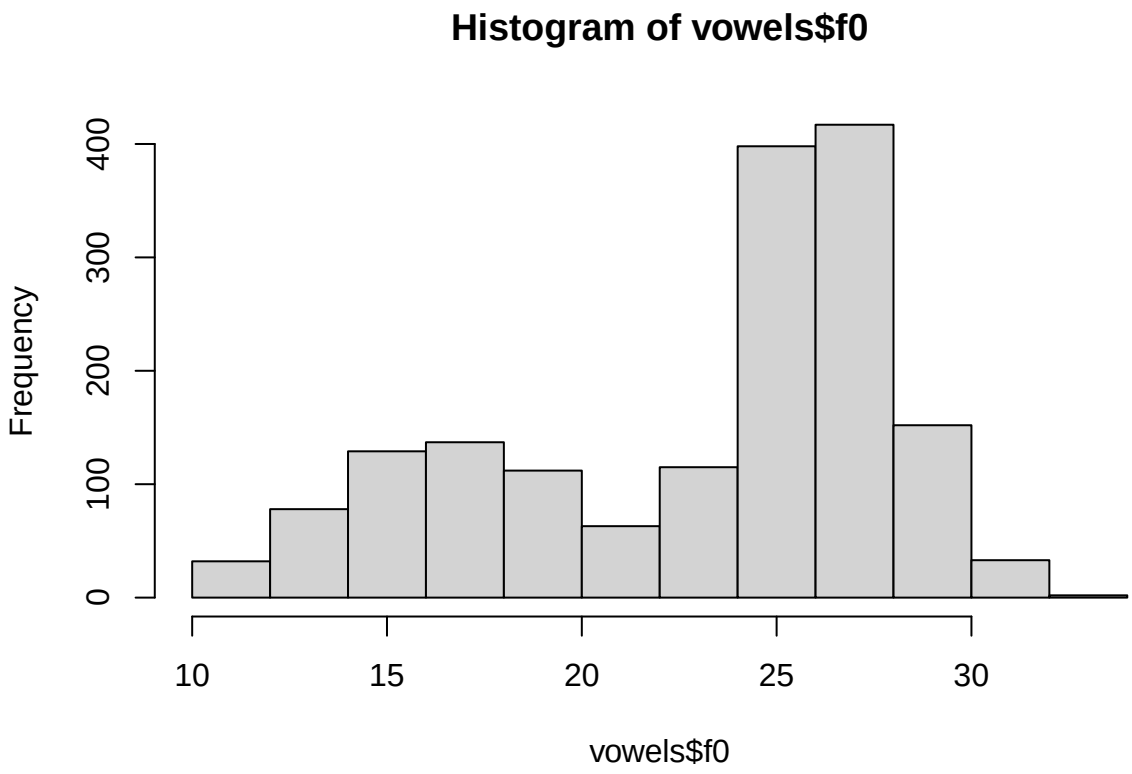
Repeat the same for F2 and front (`"iy"`) and back (`"oo"`) vowels.

```
# Write your solution here.
```

Exercise

Calculate mean f_0 for the whole data set (with all speaker groups pooled). Is this a reasonable measure of central tendency given the histogram:

```
hist(vowels$f0)
```

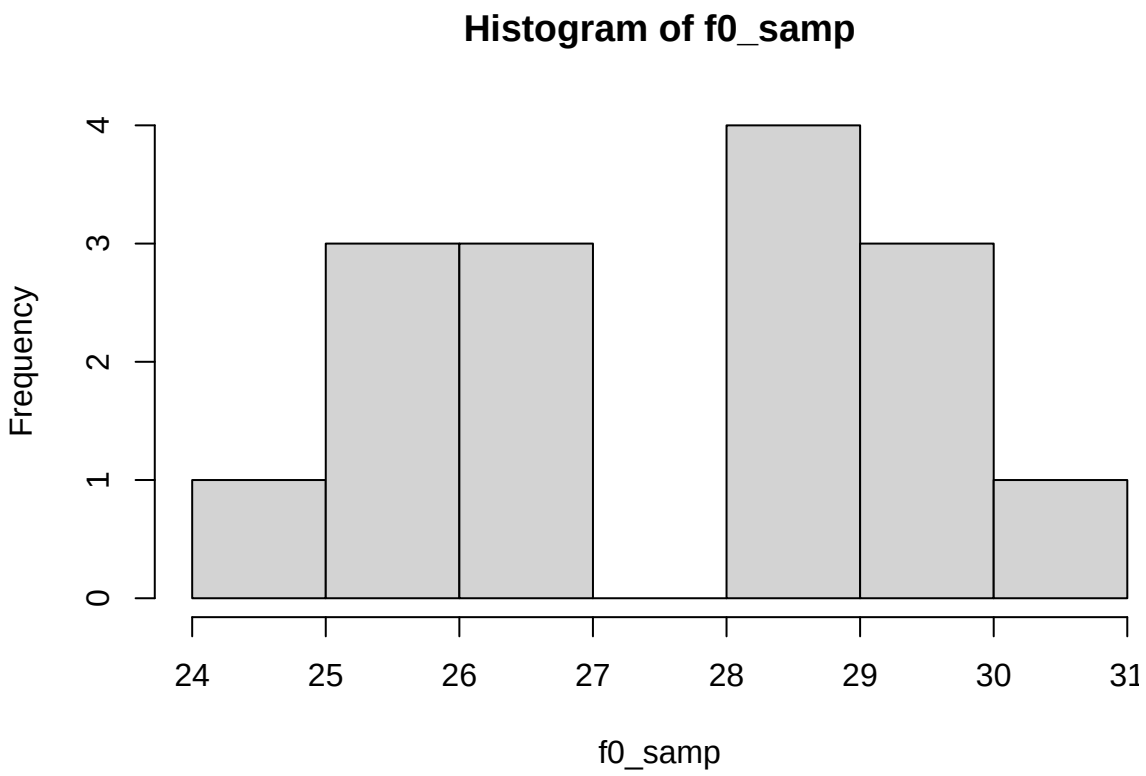



```
# Write your solution here.
```

Extreme values

To better ilustrate the effect of extreme values on the mean, we will pick at random 15 f_0 values from our sample:

```
f0_samp <- sample(f0, 15)
hist(f0_samp)
```




```
min(f0_samp)
```

```
## [1] 24.679
```

Let's now image that one of our subjects has a much lower f_0 value. We will simulate this by adding a new value which is 1.5 octaves (18 semitones) lower than the current minimum

```
new_min <- min(f0_samp) - 18
f0_samp2 <- c(f0_samp, new_min)
```

Let's see if it worked:

```
min(f0_samp)
```

```
## [1] 24.679
```

```
min(f0_samp2)
```

```
## [1] 6.679002
```

Let's calculate the mean again:

```
mean(f0_samp2)
```

```
## [1] 26.19449
```

Compared to the original value of 27.4955 the mean has changed quite a bit! This is the problem of the mean, it is very sensitive to extreme values (or outliers), that is unexpectedly large or small values. To avoid this problem, what we can use instead is the *median*, which is simply the middle value:

```
median(f0_samp)
```

```
## [1] 28.20597
```

```
median(f0_samp2)
```

```
## [1] 27.27507
```

Notice that since median only looks at the middle value, it does not matter how large the extreme value is:

```
f0_samp3 <- c(f0_samp, min(f0_samp) - 36)
median(f0_samp3)
```

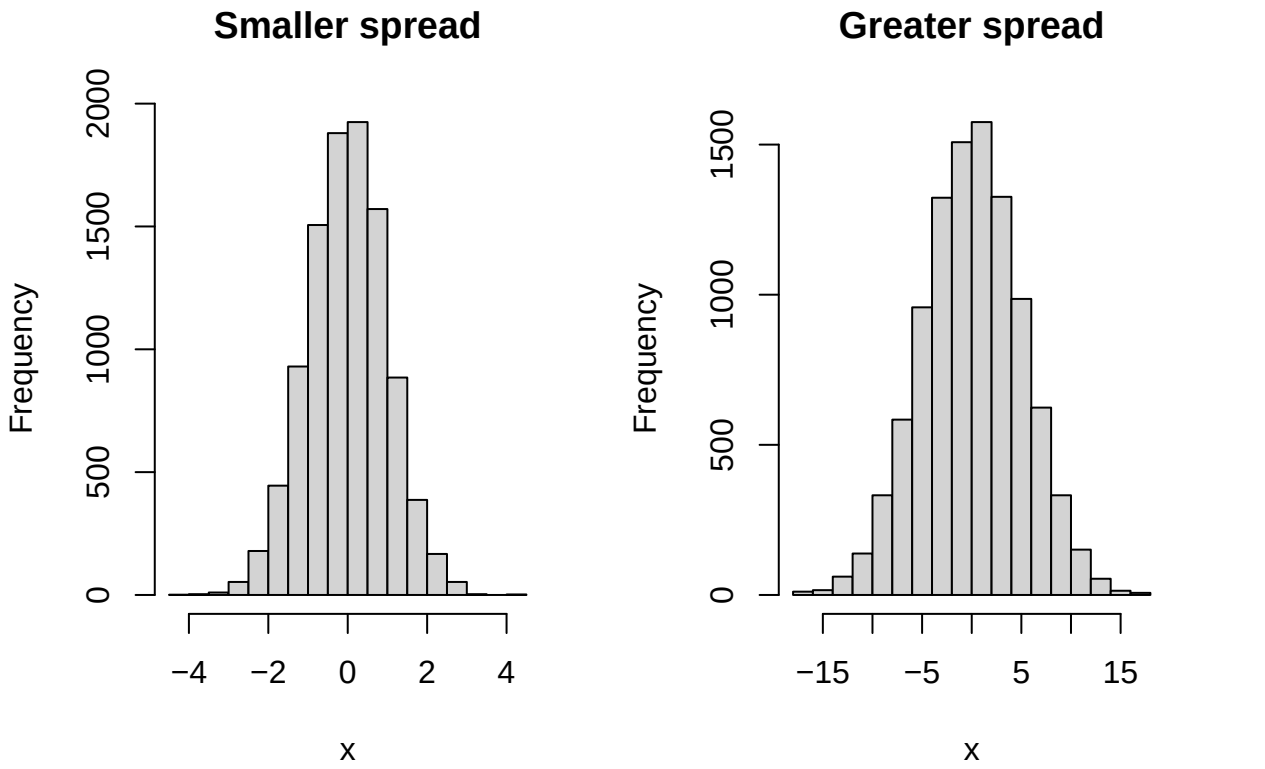
```
## [1] 27.27507
```

Measures of spread

So far we have only looked at measures of central tendency. In addition to knowing the average value, we might be also interested how spread out the values are.

For instance, in the following two distributions have the same mean (equal to 0) but the one on the right covers a much broader range of values:

```
par(mfrow=c(1, 2))
hist(rnorm(1e4, 0, 1), main = 'Smaller spread', xlab = 'x')
hist(rnorm(1e4, 0, 5), main = 'Greater spread', xlab = 'x')
```




```
par(mfrow=c(1, 1))
```

The most common-sense approach would be to calculate the *range*, i.e. the difference between the smallest and the largest elements:

```
max(f0_samp) - min(f0_samp)
```

```
## [1] 5.633823
```

However, notice that range suffers from the same problem as the mean (only even more so!) — it is very sensitive to outliers:

```
max(f0_samp2) - min(f0_samp2)
```

```
## [1] 23.63382
```

```
max(f0_samp3) - min(f0_samp3)
```

```
## [1] 41.63382
```

Instead, we would like to know how spread out (how distant from the mean) the points are *on average*.

This is what variance is good for:

$$s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2,$$

where $\bar{x}$ is the mean. It's much simpler in R:

```
var(f0)
```

```
## [1] 2.258106
```

Again, we can calculate it ourselves:

```
sum((f0 - mean(f0))^2) / (length(f0) - 1)
```

```
## [1] 2.258106
```

The result is (almost) exactly what we wanted - it is the mean of the squared distance from the mean. However, because of the power of two used in the formula, variance is expressed in *different units* than your measurement variable. For instance, we measured f_0 in semitones, but variance is expressed in semitones squared.

That's easily fixed though — all we need is to take a square root of variance. This measure is called *standard deviation*:

$$s = \sqrt[2]{s^2}$$

```
sd(f0)
```

```
## [1] 1.5027
```

We can also make sure it is indeed equal to the square root of variance:

```
sqrt(var(f0))
```

```
## [1] 1.5027
```

We can now use `sd()` with combination with `group_by` and `summarise`:

```
vowels %>%
  group_by(speaker) %>%
  summarise(f0_mean = mean(f0),
            f0_sd = sd(f0))
```

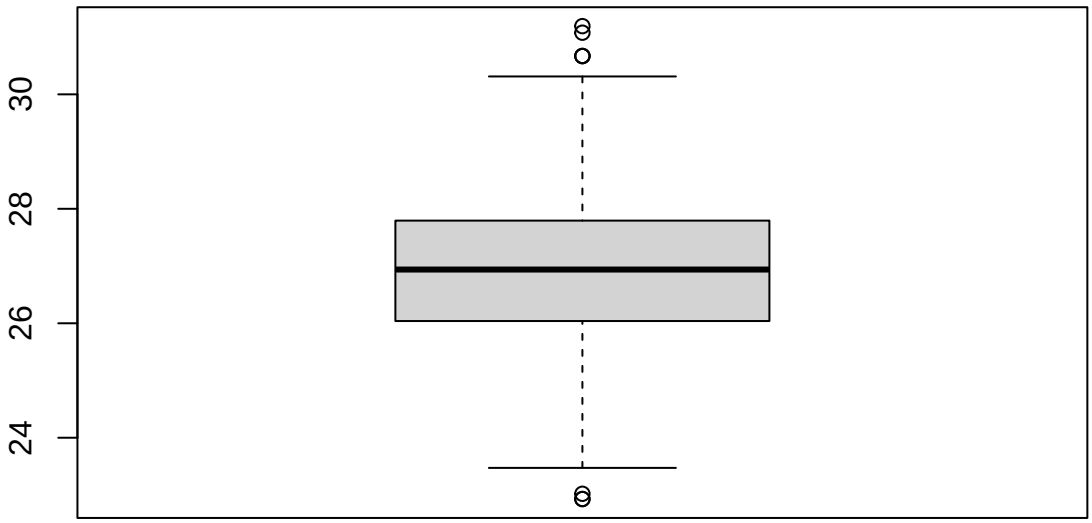
```
## # A tibble: 4 x 3
##   speaker f0_mean f0_sd
##   <chr>     <dbl> <dbl>
## 1 b          26.8  2.03
```

```
## 2 g          27.0  1.50
## 3 m          16.5  2.84
## 4 w          25.6  1.87
```

Box-and-whiskers plots

One particularly useful way of representing a sample is a box-and-whiskers plot (or boxplot, for short). You read it in the following way: the thick line in the middle of the box represents the median and the box itself represents the interval between the 25th and 75th percentiles. These are the values below which 25 and 75% of all data is found. These points are also known as the first and third quartiles and the distance between them as the interquartile range (IQR). In other words, 50% of the data lies within the interval delimited by the box. The whiskers indicate data ranges, with any outliers indicated by points outside the whiskers range.

```
boxplot(f0)
```



It is quite easy to get the numbers behind the boxplot. We already know how to calculate the median, minimum and maximum:

```
median(f0)
```

```
## [1] 26.93864
```

```
range(f0)
```

```
## [1] 22.92879 31.19181
```

To get the 25th and 75th percentiles, we need to use the `quantile` function. It takes two arguments: a numerical vector and the cut point we are interested in (in our case, 0.25 and 0.75):

```
quantile(f0, 0.25)
```

```
##     25%
## 26.0391
```

```
quantile(f0, 0.75)
```

```
##      75%
## 27.79375
```

Note that you can use the `quantile` function to get other quantiles. For instance, we can get the value below which 10% of observations lie as follows:

```
quantile(f0, 0.1)
```

```
##      10%
## 25.22783
```

Exercise

Use the `quantile` function to calculate the median.

```
# Write your solution here.
```

By default, the length of the whiskers is equal to 1.5 times the interquartile range:

```
1.5 * (quantile(f0, 0.75) - quantile(f0, 0.25))
```

```
##      75%
## 2.631973
```

or simply:

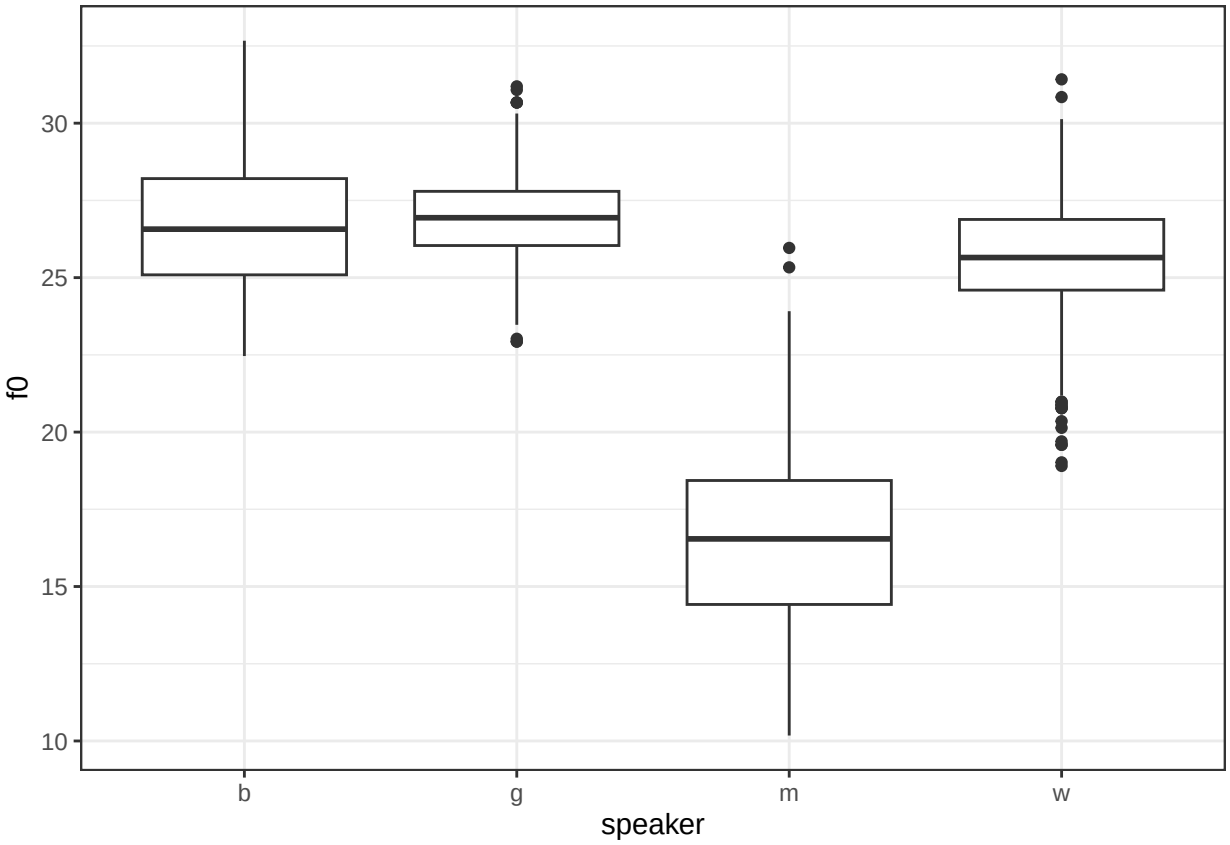
```
1.5 * IQR(f0)
```

```
## [1] 2.631973
```

Exercise

Here is how you create boxplots with `ggplot`:

```
ggplot(vowels, aes(x = speaker, y = f0)) +
  geom_boxplot()
```



Using the `vowels` data set, calculate the difference between F2 and F1 (remember to remove the rows where F2 equals 0) and save it in a new column. Then use boxplots to visualise the distribution of this feature across vowels:

```
# Write your solution here.
```